Librarian Assistant

Design Modeling & Implementation

Project Deliverable II – Part 1

Team Members

Shruthi Mallesh | Sanjeev Sreekumar Krishnan | Chenhua Fan

Course: CptS 583 – Software Quality

Institution: Washington State University

Semester: Spring 2026

Instructor: Parteek Kumar

Backend Repository: github.com/shruthi-wsu/SSC-Librarian-Assistant

Frontend Repository: github.com/sanjeevkrishnan02/library-assistant-frontend

Date: February 2026

1. Architectural Design

1.1 Architectural Pattern

The Librarian Assistant system employs a **Client-Server + Layered Architecture (n-tier) with Model-View-Controller (MVC) within the backend tier**. This combined pattern was selected because it directly maps to the full-stack nature of the project: a React Single-Page Application (SPA) acting as the client communicates over REST/JSON with a Spring Boot server that internally enforces strict layer separation.

Pattern Element	Realization in This System
Client-Server	React SPA (browser) ↔ REST/JSON ↔ Spring Boot backend on port 8081
Layered / n-Tier	Presentation → Business Logic → Data Access → Database (4 distinct layers)
MVC within Backend	REST Controllers (View/Presentation) → Service classes (Controller/Business) → JPA Repositories (Model/Data)

1.2 High-Level Architecture Diagram

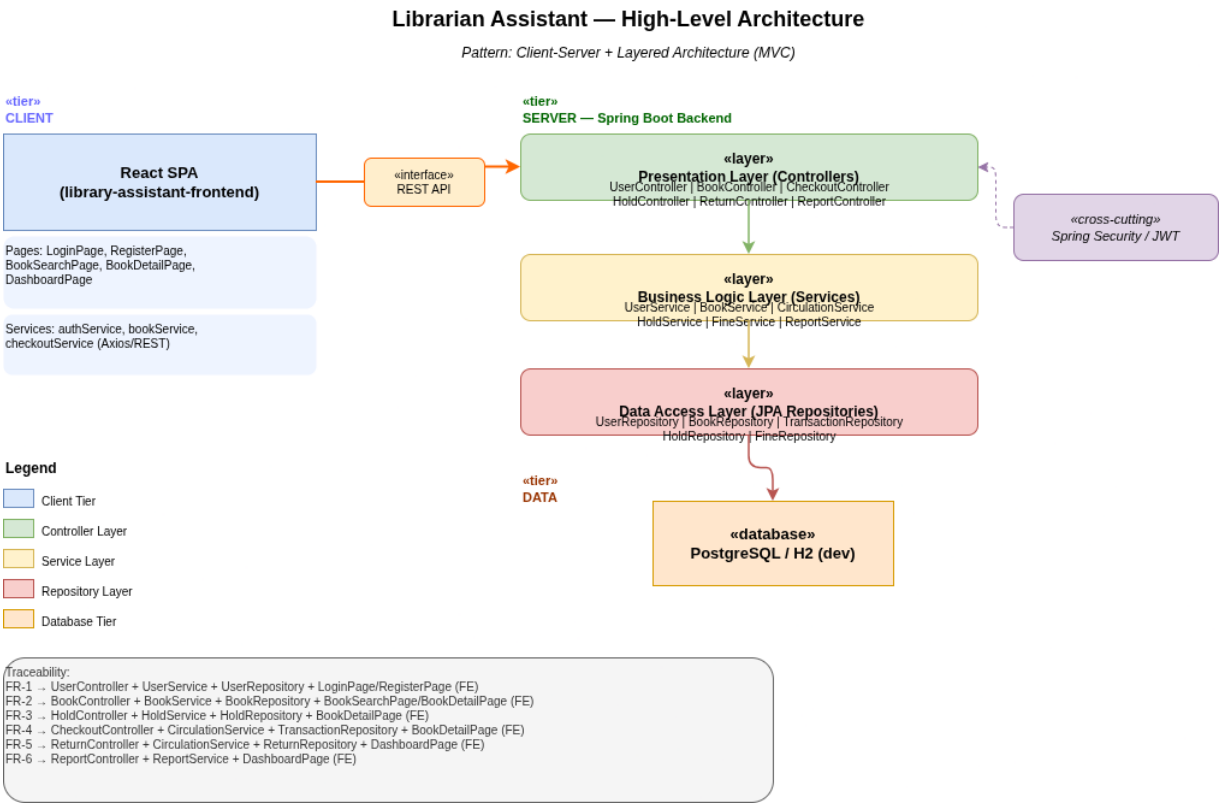

Figure 1 – High-Level Architecture: React SPA ↔ Spring Boot Layers ↔ PostgreSQL

1.3 Pattern Justification

Why Client-Server? Separating the React frontend from the Spring Boot backend allows each tier to be developed, tested, and deployed independently. The frontend team (Sanjeev) can work against mock API responses while the backend (Shruthi) exposes a stable REST contract. This separation directly supports *QG-6: Maintainability* and *QG-7: Compatibility* by enabling the UI to evolve without touching business logic.

Why Layered Architecture within the Backend? The layered model enforces a strict one-way dependency flow: Controllers depend on Services; Services depend on Repositories; Repositories depend only on JPA/DB. No layer can skip another. This produces high cohesion within each layer and controlled coupling between them — the primary design goals cited in the deliverable specification. It also enables focused unit testing at each layer, supporting *QG-6: Maintainability* (test coverage ≥ 80% target).

Why not Microservices? The team consists of three members with a fixed academic deadline. Microservices would introduce cross-service communication overhead, distributed transactions, and infrastructure complexity (service registry, API gateway) that would consume time better spent on feature quality. A well-structured monolith with clear module boundaries achieves the same separation of concerns at this scale.

Why not Server-Side Rendering (e.g., Thymeleaf)? A React SPA delivers a richer, more responsive user experience that satisfies *QG-3: Usability* (SUS ≥ 75, checkout task < 30 seconds). Server-side rendering would tightly couple the UI to the backend and complicate future mobile-client support.

1.4 Separation of Concerns

Tier / Layer	Responsibility	Technologies
Client (Frontend)	User interface, routing, form validation, JWT storage	React 18, TypeScript, Vite, Axios
Presentation (Backend)	HTTP request/response mapping, input validation, authentication filter	Spring MVC @RestController, Jakarta Validation
Business Logic	Domain rules: loan periods, fine calculation, hold queue management, RBAC	Spring @Service, Spring Security
Data Access	CRUD and query abstraction; no SQL in service layer	Spring Data JPA, @Repository interfaces
Database	Persistent storage, schema definition, referential integrity	PostgreSQL 16 (Docker container)

2. Component-Level Design

Six key components are elaborated below. Each diagram follows UML component notation created in Draw.io, showing the component name, provided interface, key attributes and operations, and inter-component dependencies.

2.1 UserManagement Component

Component: UserManagement

Provided Interface: `IUserService`

Key Attributes: userId, name, email, passwordHash, role (LIBRARIAN | PATRON), status (ACTIVE | SUSPENDED), registrationDate

Operations: registerUser(), loginUser(), getUserById(), getAllUsers(), updateUserStatus(), loadUserByUsername()

Dependencies: UserRepository, Spring Security (SecurityContext, PasswordEncoder), JwtUtil

Traces to: FR-1

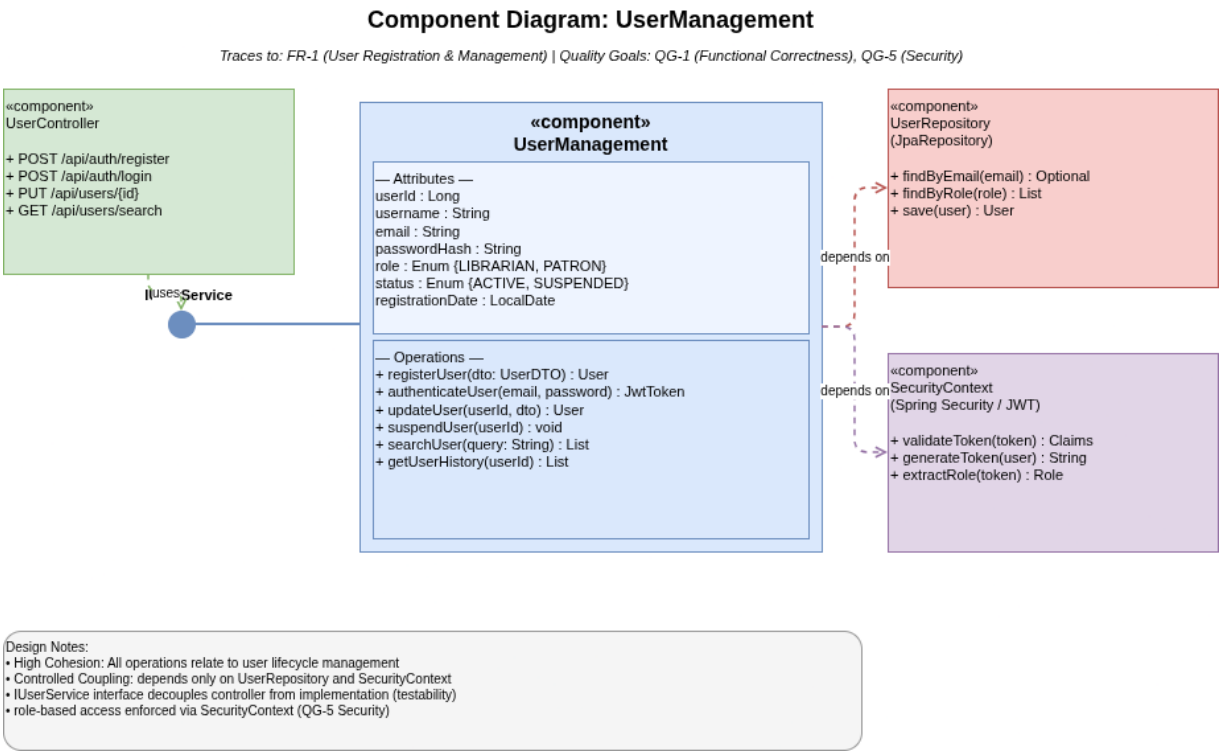

Figure 2 – UserManagement Component Diagram

The UserManagement component handles all identity and access management. It provides the `IUserService` interface consumed by the `AuthController` and `UserController`. Registration and login operations delegate to `JwtUtil` for token generation and to Spring Security's `PasswordEncoder` (BCrypt) for credential hashing. Role-based access control (LIBRARIAN vs. PATRON) is enforced at the controller layer using `@PreAuthorize` annotations backed by this component.

2.2 BookCatalog Component

Component: BookCatalog

Provided Interface: `IBookService`

Key Attributes: bookId, isbn (unique), title, author, genre, publishYear, totalCopies, availableCopies, location, status (AVAILABLE | CHECKED_OUT | LOST | DAMAGED)

Operations: addBook(), updateBook(), deleteBook(), getBookById(), getAllBooks(), searchBooks(), getAvailableBooks()

Dependencies: BookRepository

Traces to: FR-2

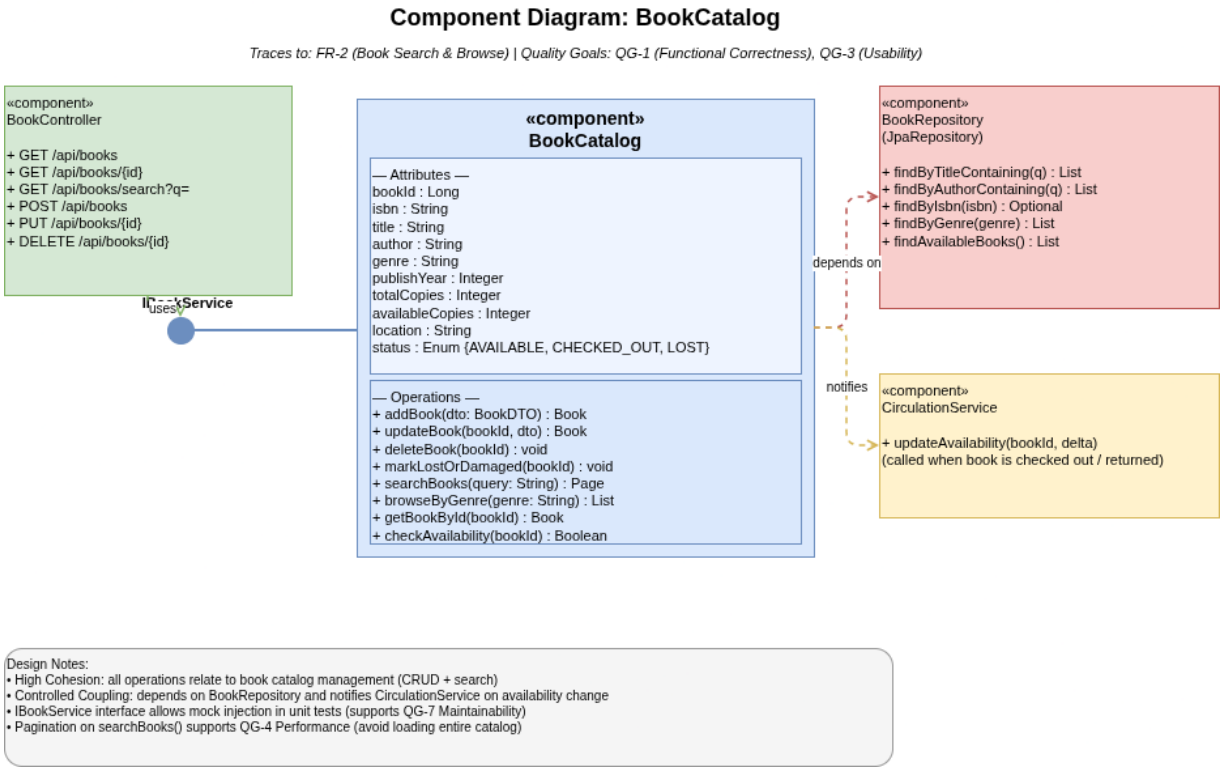

Figure 3 – BookCatalog Component Diagram

The BookCatalog component manages the complete book inventory lifecycle. Write operations (add, update, delete) are restricted to LIBRARIAN role via method-level security. The `searchBooks()`

operation uses a JPQL query performing case-insensitive matching across title, author, and ISBN fields simultaneously, supporting the patron search use case without exposing raw SQL. The `availableCopies` counter is atomically decremented/ incremented by the CirculationManagement component during checkout and return.

2.3 CirculationManagement Component

Component: CirculationManagement

Provided Interface: `ICirculationService`

Key Attributes: checkoutId, userId, bookId, checkoutDate, dueDate (14-day loan), returnDate, renewalCount (max 2), fineAmount ($0.25/day), status (ACTIVE | RETURNED | OVERDUE)

Operations: checkoutBook(), returnBook(), renewCheckout(), getUserCheckouts(), getOverdueCheckouts()

Dependencies: CheckoutRepository, BookRepository, UserRepository, FineRepository

Traces to: FR-4 FR-5

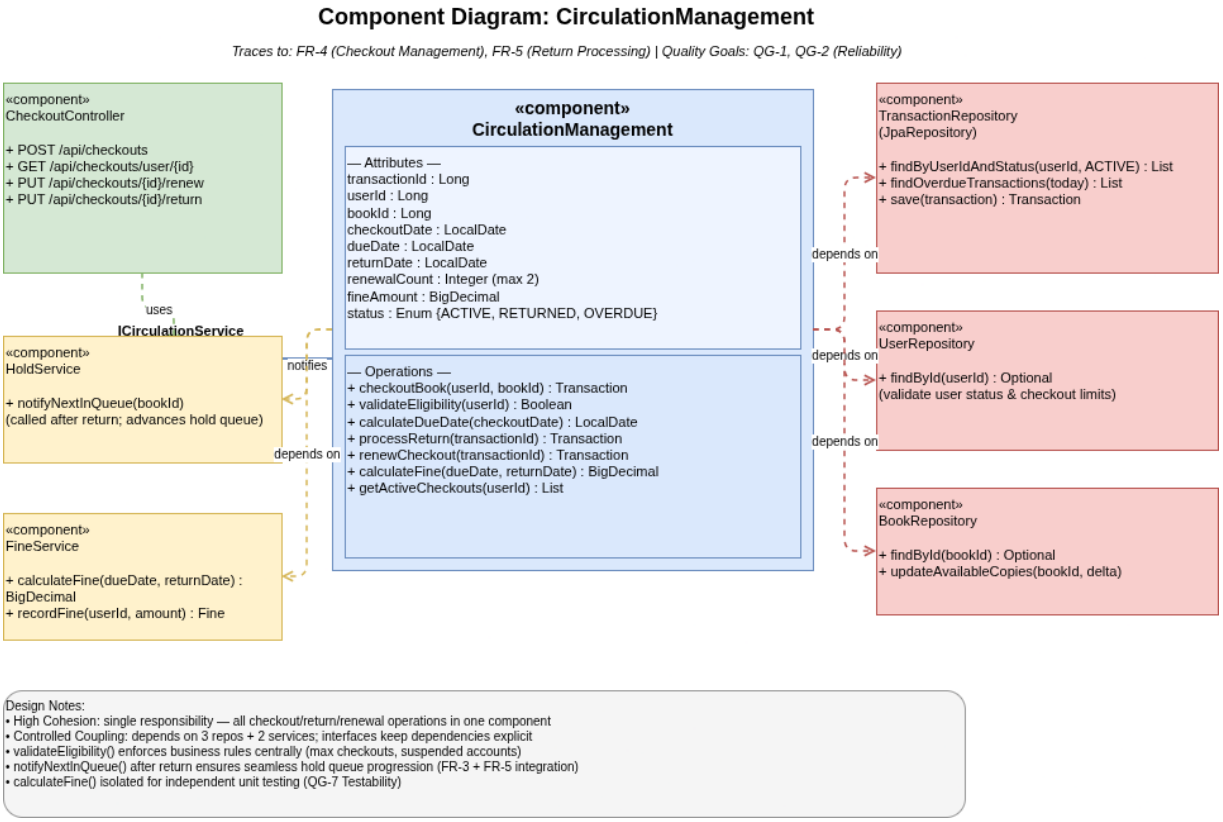

Figure 4 – CirculationManagement Component Diagram

The CirculationManagement component orchestrates the full checkout/return lifecycle within a single `@Transactional` boundary, ensuring atomicity: a failed book status update will roll back the checkout record and vice versa. Fine calculation on return is automatic — if `returnDate > dueDate`, a `Fine` entity is persisted for future payment tracking. Renewal is bounded by `MAX_RENEWALS = 2`, enforced at the service layer, ensuring fair access to popular titles.

2.4 HoldManagement Component

Component: HoldManagement

Provided Interface: `IHoldService`

Key Attributes: holdId, userId, bookId, holdDate, expirationDate, queuePosition, status (PENDING | NOTIFIED | FULFILLED | CANCELLED | EXPIRED)

Operations: placeHold(), cancelHold(), getUserHolds(), getQueuePosition(), getNextInQueue(), processExpiredHolds()

Dependencies: HoldRepository, BookRepository, UserRepository

Traces to: FR-3

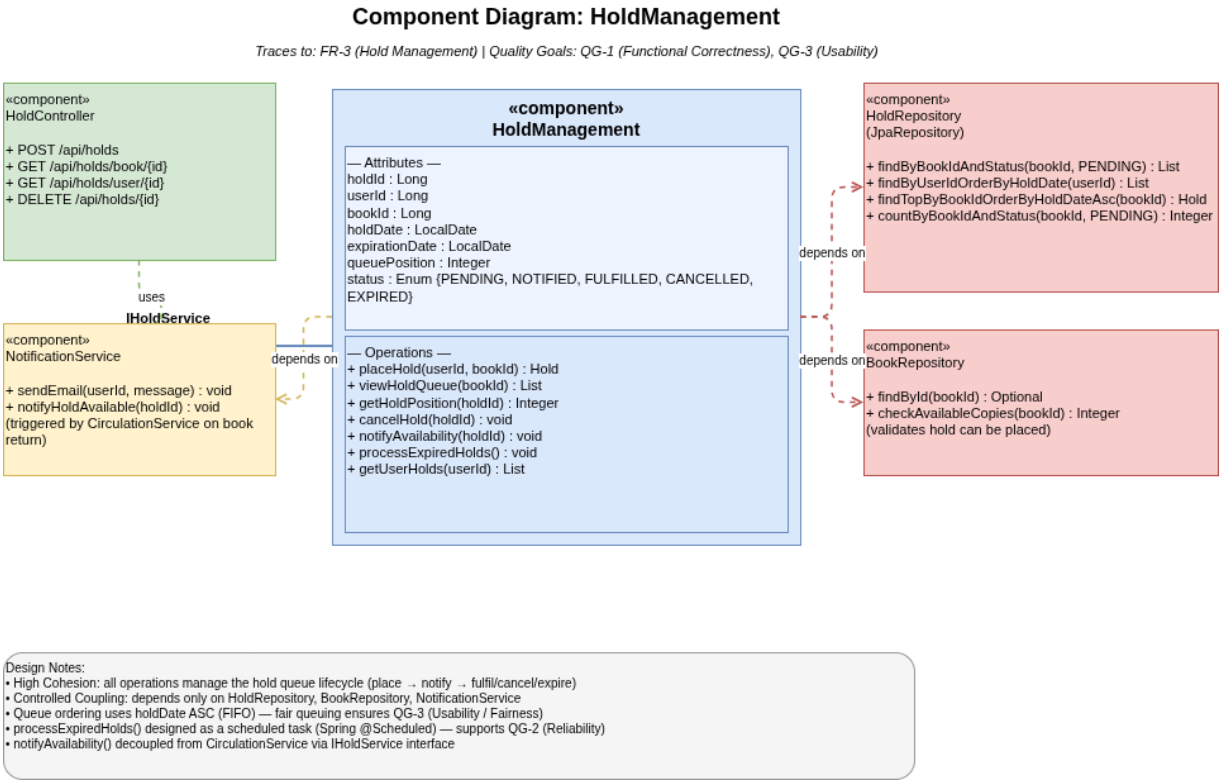

Figure 5 – HoldManagement Component Diagram

The HoldManagement component implements a FIFO hold queue keyed by book and hold date. When a patron places a hold on a checked-out book, a `Hold` record is created with `PENDING` status and the queue position is determined by the `holdDate` ordering. The repository method `findTopByBookIdAndStatusOrderByHoldDateAsc()` efficiently identifies the next eligible patron when a book is returned. Holds prevent unlimited queuing via duplicate detection (`findByUserIdAndBookIdAndStatus()`).

2.5 Frontend UI Component

Component: Frontend UI (React SPA)

Provided Interface: Browser-rendered UI; consumes REST API from Spring Boot backend

Key Attributes: authToken (JWT in localStorage), currentUser (AuthContext), searchQuery, bookList, userCheckouts

Pages / Operations: LoginPage (login, register), BookSearchPage (searchBooks, browseAll), BookDetailPage (viewBook, placeHold, checkoutBook), DashboardPage (viewMyCheckouts, returnBook)

Shared Components: Navbar, BookCard, SearchBar

Services: authService.ts, bookService.ts, checkoutService.ts (Axios wrappers)

Dependencies: Spring Boot REST API (HTTP/JSON), React Router (navigation), Axios (HTTP client)

Traces to: FR-1 FR-2 FR-3 FR-4 FR-5 FR-6

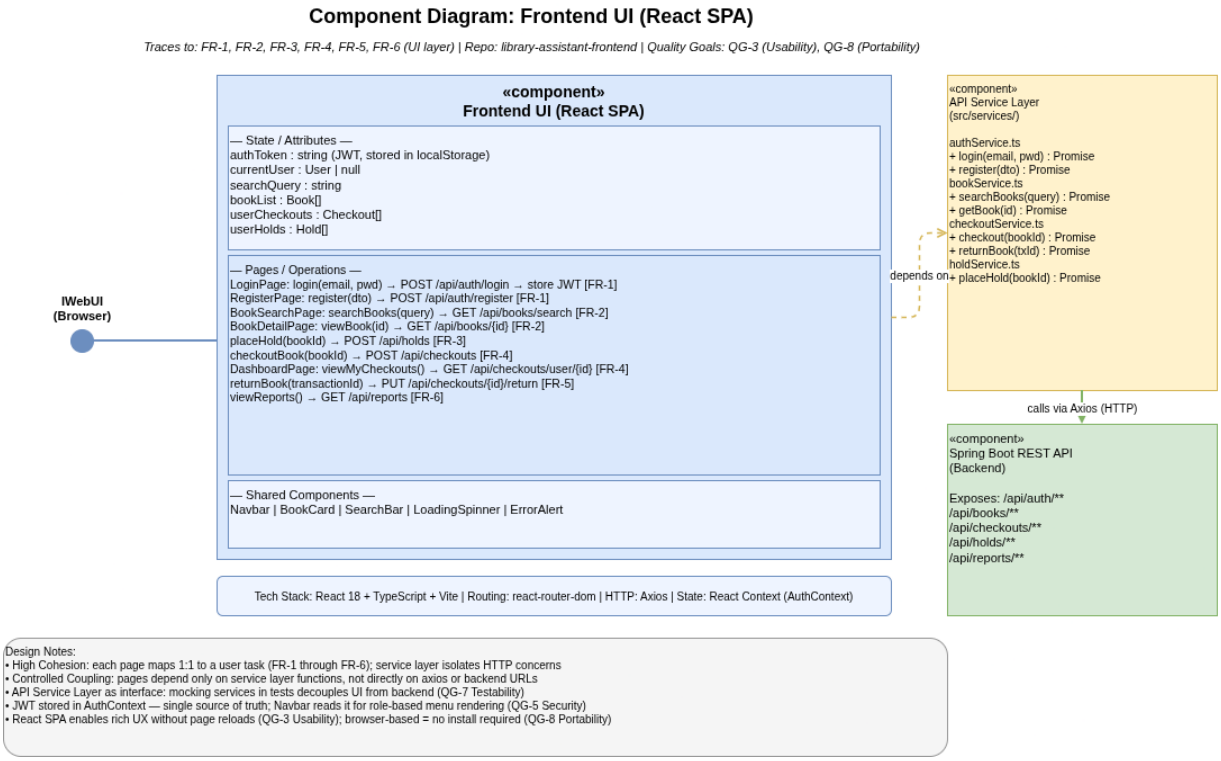

Figure 6 – Frontend UI Component Diagram (React SPA)

The Frontend UI component is the sole entry point for end users. It is architecturally isolated from the backend — all communication passes through typed Axios service wrappers that construct REST requests and parse JSON responses. The `AuthContext` React context stores the JWT token and current user role, making authentication state globally accessible. React Router enforces route-level authorization (patron routes vs. librarian routes) on the client side, complementing the server-side `@PreAuthorize` guards. Vite's dev-proxy configuration routes `/api/*` calls to `http://localhost:8081`, eliminating CORS issues during development.

2.6 Requirement Traceability Table

Functional Req.	Backend Component	Frontend Page	Interface	Quality Goals
FR-1: User Management	UserManagement	LoginPage, RegisterPage	IUserService / `/api/auth/**`	QG-1, QG-5
FR-2: Book Catalog	BookCatalog	BookSearchPage, BookDetailPage	IBookService / `/api/books/**`	QG-1, QG-3
FR-3: Hold Management	HoldManagement	BookDetailPage	IHoldService / `/api/holds/**`	QG-1, QG-3
FR-4: Checkout	CirculationManagement	BookDetailPage	ICirculationService / `/api/checkouts`	QG-1, QG-2
FR-5: Return & Renewal	CirculationManagement	DashboardPage	ICirculationService / `/api/checkouts/{id}/return`	QG-1, QG-2
FR-6: Reporting	ReportingAnalytics (planned)	DashboardPage	IReportService / `/api/reports/**`	QG-1, QG-4

3. Design Reasoning & Trade-offs

3.1 Architectural Pattern Decision

The selection of **Client-Server + Layered + MVC** was driven by three primary forces:

1. **Team Structure:** Three developers with distinct frontend, backend, and infrastructure responsibilities. The client-server split provides a natural work boundary with the REST API as the contract.
2. **Quality Goals:** QG-5 (Security) requires all business rules and data validation to live in a server-side layer unreachable by client-side manipulation. The layered backend enforces this: the Service layer validates all domain rules before any data reaches the Repository.
3. **Testability:** Layered architecture enables isolated unit tests for each layer with mocked dependencies — a direct prerequisite for achieving the QG-6 Maintainability target of ≥ 80% test coverage (JaCoCo).

3.2 Alternatives Considered

Alternative	Reason Rejected
Microservices (separate services per domain)	Prohibitive operational complexity for a 3-person academic team; distributed transactions across checkout/fine/hold services would introduce consistency challenges that are out of scope for this course.
Monolith with Server-Side Rendering (Thymeleaf/JSP)	Tightly couples UI design to backend release cycles; limits future mobile app development; does not satisfy QG-3 Usability requirements for a rich, responsive patron experience.
Event-Driven Architecture (Kafka/RabbitMQ)	Unnecessary overhead for a single-instance library system; adds infrastructure dependencies that would reduce portability (QG-8) and increase deployment complexity beyond the scope of this project.
Flat / Anemic Architecture (no service layer)	Business rules (fine calculation, renewal limits, hold queue ordering) would bleed into controllers, making the code untestable and violating the separation-of-concerns requirement in the grading rubric.

3.3 Key Design Trade-offs

Decision	Benefit	Trade-off Accepted
Stateless JWT authentication	Scales horizontally; no server-side session storage; supports the React SPA cleanly	Token revocation requires additional infrastructure (token blacklist) — deferred to future sprint
PostgreSQL via Docker	Reproducible environment; no system-level installation conflicts; consistent behavior across developer machines	Requires Docker to be running; H2 used for unit tests to avoid container dependency in CI
Spring Data JPA (repository pattern)	Eliminates boilerplate SQL; Spring generates JPQL from method names; type-safe queries	Less control over query optimization for complex reporting queries — custom `@Query` used selectively for overdue lookups
`ddl-auto=create` during development	Schema auto-updated on each restart; no migration scripts needed at this stage	Data is lost on restart — DataSeeder repopulates test data; production will switch to `validate` + Flyway
@Transactional boundaries at service layer	Atomic checkout/return operations: book availability and checkout status change together or not at all	Slightly larger transaction scope than strictly necessary, but acceptable at this scale

3.4 Requirement Revisions Triggered by Design

During design modeling, one implicit requirement was made explicit:

- **Security layer requirement (implicit → explicit):** The original FR-1 described "user login" without specifying token mechanism. Design analysis revealed that a stateless REST API serving a SPA client requires explicit JWT issuance, refresh handling, and a `JwtAuthFilter` for every protected endpoint. This was formally documented as a security sub-requirement and implemented in the `SecurityConfig` and `JwtAuthFilter` classes.
- **Fine tracking (clarified):** FR-5 (Return) implied fine calculation but did not explicitly require a persistent `Fine` entity. The component design identified that reporting (FR-6) and patron

payment workflows require persisted fine records — hence the `Fine` JPA entity and `FineRepository` were added as part of the data schema.

4. Quality Alignment Summary

The following table maps each quality goal (defined in Deliverable I) to the specific architectural and design decisions that support it.

Quality Goal	Priority	Supporting Design Decision	Metric / Evidence
QG-1: Functional Correctness	Critical	Service layer owns all domain rules (fine = $0.25/day, max renewals = 2, FIFO hold queue). Rules are centralized, testable, and cannot be bypassed by the controller layer.	Unit tests per service class; target: zero critical defects in production
QG-2: Reliability	Critical	@Transactional boundaries ensure atomic checkout/return operations. PostgreSQL ACID guarantees prevent partial updates. Docker container with restart policy ensures DB availability.	MTBF > 720 hours; zero data loss target
QG-3: Usability	High	React SPA delivers a responsive, client-side-rendered UI with instant feedback. Separate pages per workflow (LoginPage, BookSearchPage, DashboardPage) minimize cognitive load.	SUS score ≥ 75; checkout task < 30 seconds
QG-4: Performance Efficiency	High	JPQL search query with index-friendly LIKE patterns. HikariCP connection pool. Stateless JWT eliminates session-store round-trips. H2 in-memory DB used in tests for speed.	95th pct response ≤ 2 s; DB search ≤ 1 s
QG-5: Security	High	BCrypt password hashing. HMAC-SHA256 JWT tokens (jjwt 0.12.6). Stateless session (STATELESS policy). @PreAuthorize RBAC on all write endpoints. CORS restricted to frontend origin.	Zero unauthorized access; 100% PII encrypted in transit (HTTPS in production)
QG-6: Maintainability	Medium	Layered architecture enables layer-isolated testing with Mockito. Lombok reduces boilerplate. Conventional Commits enforce readable history. Feature-branch GitHub workflow enables code review.	Test coverage ≥ 80% (JaCoCo); cyclomatic complexity < 10
QG-7: Compatibility	Medium	React SPA tested on Chrome/Firefox/Safari. Standard REST/JSON API is client-agnostic. PostgreSQL Docker container works on Windows, macOS, and Linux.	Runs on latest 2 versions of major browsers

QG-8: Portability	Low	Docker Compose file bundles PostgreSQL for one-command setup. Gradle wrapper eliminates JDK version ambiguity. GitHub Actions CI validates builds on every push.	Installation time < 2 hours; 3+ environment deployment verified

4.1 Workflow Quality Practices

Practice	Implementation	Supports
Feature Branch Workflow	Every issue has a `feature/<issue-id>-<description>` branch; merges via PR to `development`	QG-6 Maintainability, Engineering Discipline
Conventional Commits	`feat(#13): ...`, `fix(#11): ...` — machine-readable history	QG-6, Process Discipline
GitHub Projects Kanban	Issues move: Backlog → In Progress → Done; sprint-tagged, priority-labeled	Engineering Discipline, Visibility
CI Pipeline (GitHub Actions)	`./gradlew build && ./gradlew test` on every push and PR to main/development	QG-1 Correctness, QG-6 Maintainability
H2 In-Memory Test DB	`src/test/resources/application.properties` overrides PostgreSQL with H2 for fast, dependency-free CI tests	QG-6, QG-4 Performance in pipeline

4.2 Next Steps Toward Full Quality Assurance

- **Sprint 2:** Add JaCoCo Gradle plugin; generate coverage report; target ≥ 80%
- **Sprint 3:** Integrate SonarQube for static analysis; resolve all critical code smells
- **Sprint 3:** Complete integration test suite (Spring Boot test slices + Testcontainers for PostgreSQL)
- **Sprint 4:** JMeter load testing for NFR-1 (10 concurrent sessions, ≤ 2 s p95 response)
- **Sprint 4:** Switch `ddl-auto` to `validate` and add Flyway migrations for production safety
- **Sprint 4:** Docker Compose production profile with HTTPS (Let's Encrypt / self-signed)